# TP Apprentissage profond pour la vision

Roméo Agostino

## Questions 1 et 2

Le code source propose plusieurs méthodes d'apprentissage auto-supervisé. Le TP se concentre sur les trois premières :

1. **Context Encoder** : une partie de l'image est masquée et le modèle apprend à reconstruire la zone manquante à partir du contexte.
2. **RotNet** : l'image subit une rotation et le modèle apprend à prédire l'angle appliqué.
3. **SimCLR** : deux versions augmentées d'une même image sont générées et le modèle compare leurs représentations.

## Questions 3 et 4

Pour comparer les trois méthodes, j'ai utilisé la fonction `evaluate_classifier()` fournie, qui calcule l'accuracy sur les ensembles d'entraînement et de test. J'ai réalisé l'évaluation sur CIFAR-10, puis sur CINIC-10 (270 000 images $32 \times 32$ réparties en 10 classes) pour la question 4.

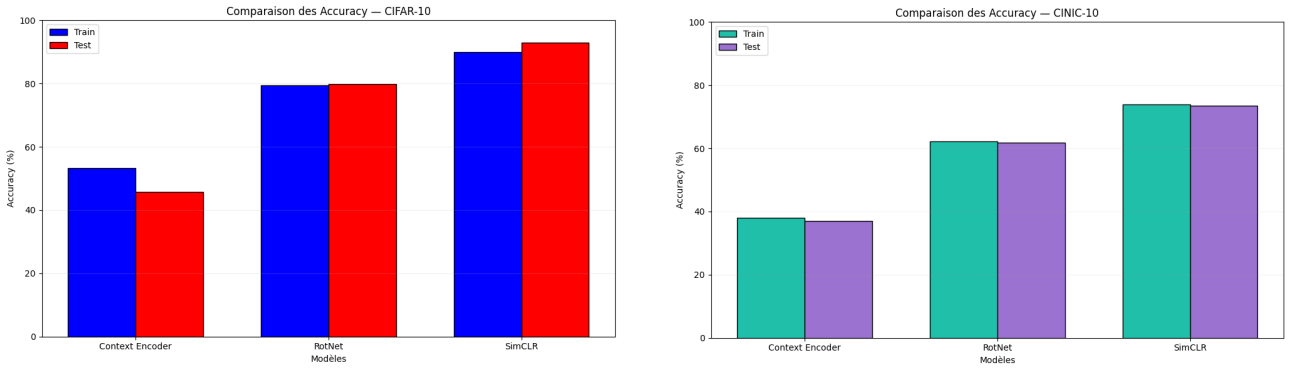



FIGURE 1 – Accuracy des trois méthodes en train et en test : CIFAR-10 (gauche) et CINIC-10 (droite)

Sur CIFAR-10, SimCLR obtient la meilleure performance (**92,84 %** en test, 90,1 % en train), suivi de RotNet (**79,91 %** / 79,66 %) et de Context Encoder (**45,77 %** / 53,24 %). Sur CINIC-10, la hiérarchie est identique : SimCLR (**73,50 %** / 73,87 %), RotNet (**61,83 %** / 62,25 %) et Context Encoder (**36,98 %** / 38,04 %). Toutes les méthodes perdent en performance sur CINIC-10, car les images sont différentes de celles de CIFAR-10.

## Question 5

J'ai implémenté avec PyTorch une tâche prétexte consistant à prédire la position relative d'un patch B par rapport à un patch de référence A, sur CIFAR-10. Chaque image est découpée en une grille de $3 \times 3$ patchs de $10 \times 10$ pixels. Un patch A est tiré aléatoirement, puis un patch B parmi ses voisins ; la position de B par rapport à A constitue le label, parmi 8 classes. La tâche est auto-supervisée : les labels sont générés automatiquement, sans utiliser les classes de CIFAR-10.

**Architecture :** Les patchs A et B passent séparément dans un même encodeur CNN à poids partagés : trois convolutions (32, 64 et 128 filtres) avec ReLU, un MaxPooling, puis un Flatten. Les deux vecteurs de caractéristiques sont concaténés et une couche linéaire produit 8 scores.

**Entraînement :** J'ai utilisé les 50 000 images d'entraînement, des batchs de 128, la CrossEntropyLoss et l'optimiseur Adam (learning rate 0,001). À chaque epoch, je calcule la loss moyenne et l'accuracy.

| Architecture | Epochs | Loss train | Accuracy train |
|---|---|---|---|
| 3 convolutions (32, 64, 128) | 5 | 1,8592 | 29,47 % |
| 3 convolutions (32, 64, 128) | 10 | 1,8179 | 31,04 % |

Table 1 – Résultats selon la durée d'entraînement

Passer de 5 à 10 epochs améliore légèrement les résultats, mais la progression ralentit. Sur les 10 000 images de test, j'obtiens 29,69 % d'accuracy.

**Analyse :** Le modèle apprend bien certaines relations spatiales : il dépasse nettement le hasard. Les résultats restent toutefois limités, ce que j'explique surtout par la petite taille des patchs ($10 \times 10$), qui contiennent parfois trop peu d'information visuelle. L'écart faible entre train et test montre une bonne généralisation.

## Question 6

J'ai comparé mon modèle aux trois autres méthodes par *Linear Probing* : j'ai gelé l'encodeur entraîné à la question 5 et remplacé le classifieur à 8 positions par une couche linéaire à 10 classes, entraînée seule pendant 5 epochs (Adam, learning rate 0,001). J'obtiens 78,54 % en entraînement et 64,93 % sur CIFAR-10 Test. Évalué sans réentraînement sur CINIC-10, le modèle atteint 52,06 %.

| Modèle | CIFAR-10 Test | CINIC-10 Test |
|---|---|---|
| Context Encoder | 45,77 % | 36,98 % |
| RotNet | 79,91 % | 61,83 % |
| SimCLR | 92,84 % | 73,50 % |
| Relative Patch Position | 64,93 % | 52,06 % |

Table 2 – Comparaison des quatre méthodes sur CIFAR-10 et CINIC-10

SimCLR reste la meilleure méthode sur les deux datasets, suivie de RotNet, mon modèle arrive troisième, devant Context Encoder. Tous les modèles perdent en performance sur CINIC-10. Même s'il est moins performant que SimCLR et RotNet, mon modèle montre que la prédiction de positions relatives permet d'apprendre des caractéristiques utiles à la classification. Mais cette comparaison reste à nuancer car les architectures et les préentraînements sont différents selon les méthodes.